

CP5-DEVOPS

DimDim Pedidos - Java e Banco em Nuvem

Gabriel Sbrana Campos - RM565849

Thiago Rodrigues da Mota - RM563650

Moisés Waidemann Molinillo - RM563719

Aplicação de gestão de pedidos e itens, com CRUD, duas tabelas relacionadas no Azure SQL, implantação pelo Azure CLI e telemetria real no Application Insights.

Repositório e How To

github.com/sbranaw/CP5-DEVOPS

Documentos: README.MD (explicação completa), docs/guia-execucao.md (requisitos e caminhos), docs/how-to.md (implantação e validação).

Aplicação publicada

dimdim-pedidos-261002-ce05.azurewebsites.net/pedidos

Relatório preparado em 2 de outubro de 2026. As evidências abaixo foram registradas nas etapas de desenvolvimento. Conferir acesso ao site e ao repositório antes da entrega.

Arquitetura e dados

Navegador → controlador Spring MVC → serviço transacional → repositório JPA → Azure SQL. Thymeleaf renderiza as telas; o agente Java do App Service coleta requisições e chamadas ao banco.

Componente	Implementação
Linguagem / framework	Java 21 / Spring Boot 4.0.8
Build	Maven Wrapper 3.9.15
Banco	Azure SQL Basic
Hospedagem	App Service Linux F1
Monitoramento	Application Insights / Logs

Modelo master-detail

dbo.pedido: id (PK), nome_cliente, data_pedido e status.

dbo.item_pedido: id (PK), pedido_id (FK), descricao_produto, quantidade e preco_unitario. Relação 1:N, com ON DELETE CASCADE. DDL: database/001-create-tables.sql.

Regras

Pedido começa RASCUNHO; só rascunhos podem ser editados. Confirmação exige item e bloqueia edição. Cancelamento preserva dados. Exclusão confirmada remove o pedido e os itens. BigDecimal calcula quantidade × preço e soma os subtotais sem gravar totais no banco.

Esta versão utiliza formulários HTML com POST e CSRF, sem login e sem API REST JSON. O perfil demo usa H2 temporário; a publicação usa Azure SQL e valida o esquema sem criá-lo automaticamente.

Execução e implantação

Requisitos locais: JDK 21, terminal e internet inicialmente. Maven está incluído pelo Wrapper. Na pasta com pom.xml:

```
./mvnw.cmd verify
java -jar target/dimdim-pedidos-0.1.0.jar
--spring.profiles.active=demo
```

A linha do comando Java acima está quebrada apenas para leitura: execute-a em uma única linha. Abra <http://127.0.0.1:8080/pedidos>. Ctrl+C encerra; dados demo desaparecem.

Persistência local Azure SQL

Configure DB_URL, DB_USERNAME e DB_PASSWORD na sessão e execute o mesmo JAR sem perfil demo. Prepare DDL, permissões CRUD e firewall TCP 1433. Leia a senha de forma oculta, conforme o guia. Não versionar senhas, tokens ou .local.

Azure CLI

Requisitos: PowerShell 7, Azure CLI, az login e assinatura com permissões. Configuração pública de exemplo: infra/azure.example.json. Configuração preenchida: .local/azure-config.json.

Deploy-Azure.ps1 mostra o plano; com -Execute provisiona grupo, plano, Web App, SQL, firewall, DDL, usuário CRUD, testes e JAR. Test-AzureDeployment.ps1 valida o destino. Enable-ApplicationInsights.ps1 configura o agente e Test-ApplicationInsights.ps1 comprova ingestão.

Recursos reais

Grupo: rg-dimdim-261002-ce05
Web App: dimdim-pedidos-261002-ce05
SQL: dimdim-sql-261002-ce05 / dimdim-pedidos
Insights: ai-dimdim-pedidos-261002-ce05
Região: Brazil South

SQL Basic e logs podem consumir créditos. Workspace: retenção 30 dias e cota diária 0,1 GB, sem garantia de teto de custo.

Interface publicada

D

DimDim

Pedidos

Todos os pedidos

Novo pedido

Item adicionado.

← Todos os pedidos

PEDIDO #3

Cliente de demonstração Azure

Rascunho

Criado em 02/10/2026

Editar pedido

Confirmar pedido

Cancelar pedido

[Excluir pedido](#)

Itens do pedido

+ Adicionar item

Produto	Quantidade	Preço unitário	Subtotal	
Teclado sem fio	2	R\$ 149,90	R\$ 299,80	Editar Exc

Total do pedido **R\$ 299,80**

Captura real da etapa 4: aplicação publicada e pedido cadastrado pela interface. O registro foi conferido diretamente no Azure SQL. Os relatórios completos estão em docs/resultado-etapa-4.md e docs/resultado-etapa-5.md.

Validações e telemetria

Verificação registrada	Resultado
Testes locais	44 aprovados
Integração Azure SQL real	2 aprovados
GET /pedidos HTTP 200	11 registros
Dependências SQL	16, sem falhas
GET /pedidos/0 HTTP 404	1 teste controlado

Os testes SQL verificaram CRUD e a cascata do DDL. Os valores de telemetria correspondem à janela consultada na etapa 5; não representam monitoramento contínuo de disponibilidade.

Evidências: docs/telemetria-etapa-5.json e docs/respostas-http-etapa-5.json. Um segundo 404 foi identificado como o aquecimento padrão do App Service (/robots933456.txt), não como falha SQL.

Consultas: monitoring/requests.kql, sql-dependencies.kql e http-errors.kql. A validação confirmou ingestão real, não apenas configurações presentes. Não foram criados alertas.

Referência oficial

[Microsoft: configurações e aquecimento do App Service](#)

Roteiro final e penalidades

1. Mostrar fontes, How To, DDL e scripts CLI no GitHub.
2. Criar pedido e conferir `dbo.pedido`.
3. Adicionar dois itens e conferir `dbo.item_pedido/FK`.
4. Consultar detalhes, editar cliente e item, conferindo o banco.
5. Excluir item e conferir ausência da linha.
6. Confirmar/cancelar e mostrar regras de status.
7. Excluir pedido e conferir ausência de pedido e itens.
8. Mostrar requisições e SQL no Application Insights.

Conferir o banco após cada operação. Use ID real da demonstração no `Inspect-AzurePedido.ps1` ou em `SELECT` nas duas tabelas. Preserve o pedido de exemplo. Não expor credenciais.

Falha	Desconto
Entrega fora do padrão	0,5 ponto
Não mostrar operações no banco	2 pontos
Sem Application Insights	1 ponto
Sem How To no GitHub	1,5 ponto
Sem DDL	1 ponto
Apenas uma tabela	2,5 pontos

Antes de entregar: confirmar acesso do avaliador ao GitHub, demonstrar CRUD no banco, revisar links e PDF. Respeitar Azure CLI e a proibição de reutilizar checkpoints anteriores/Sprint 3. O representante faz upload de `CP5-DEVOPS_webapp.pdf` no Teams.